Experiment-2

Feedforward Neural Network (MLP)

1. Aim

To understand the architecture and training process of a Multilayer Perceptron (MLP) for tabular data by implementing it on the Iris dataset, with detailed visualization of forward propagation, backpropagation, and hidden-layer activations for selected samples.

2. Theory

Introduction to Feedforward Neural Networks

A Multilayer Perceptron (MLP) is a class of feedforward neural network, designed to learn a mapping between input data and corresponding output targets, expressed as

$$\hat{y} = f(x; \theta),$$

where x represents the input features, $\hat{y}$ denotes the predicted output, and θcomprises the model parameters, including weights and biases. In classification tasks, the model assigns inputs to discrete classes, while in regression it predicts continuous values. A feedforward network is characterised by the unidirectional flow of information from the input layer through one or more hidden layers to the output layer, without any feedback connections, making it suitable for standard prediction tasks.The objective of such networks is to approximate an underlying target function f^*. For instance, in a classification task, the network learns a mapping

$$y = f^*(x),$$

in classification problems. A layered feedforward network ensures that all paths from input to output pass through the same number of layers, and it is considered fully connected when each neuron in one layer is linked to every neuron in the next. The key strength of MLPs lies in their ability to model complex, non-linear relationships and to generalise effectively to unseen data when trained on representative datasets.

These models are called feedforward because information flows through the function being evaluated from x, through the intermediate computations used to define f, and finally to the output $\hat{y}$. There are no feedback connections in which the outputs of the model are fed back into itself.

I. **Gradient-Based Learning:** For feedforward neural networks, it is important to initialise all weights to small random values; biases may be initialised to zero or to small positive values. Iterative gradient-based optimisation algorithms, such as SGD, RMSProp, and Adam, are used to train feedforward networks and deep models.

II. **Learning XOR:** To illustrate the capabilities of feedforward networks, consider the XOR function. XOR returns 1 when exactly one of x_1 or x_2 is 1, and 0 otherwise. Learning XOR

demonstrates that an MLP with a hidden layer can represent non-linearly separable functions.

To make the idea of a feedforward network more concrete, we begin with an example of a fully functioning feedforward network on a very simple task: learning the XOR function. The XOR function ("exclusive or") is an operation on two binary values, x_1 and x_2. When exactly one of these binary values is equal to 1, the XOR function returns 1. Otherwise, it returns 0. The XOR function provides the target function $y = f^*(x)$ that we want to learn. Our model provides a function $y = f(x; \theta)$, and our learning algorithm adapts the parameters θ to make f as similar as possible to f^*.

Architecture of MLP

An MLP consists of multiple layers arranged sequentially:

- **Input layer:** accepts raw input features.
- **Hidden layers:** perform intermediate transformations.
- **Output layer:** produces the final prediction.

Each neuron in a layer is connected to every neuron in the next layer, forming a fully connected network. Hidden layers are responsible for learning useful intermediate representations from the data.

Up to now, neural networks have been described as models in which the output of one layer is used as the input to the next layer; such architectures are called feedforward neural networks. In a pure feedforward network, information is always fed forward, never fed back, so there are no recurrent feedback loops. This can also be shown in Figure 1.

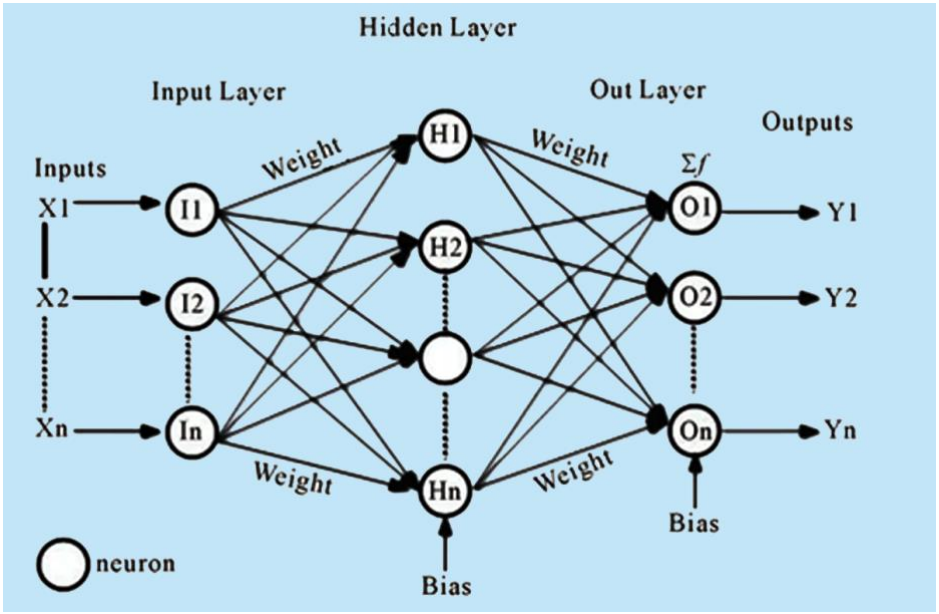

Figure 1 – Feed-Forward Neural Network Architecture
(Source: Based on the standard multilayer perceptron model as presented in Goodfellow, Bengio & Courville, Deep Learning, MIT Press, 2016)

Mathematical Representation of MLP

Let the network consist of Llayers. The computation at each layer is performed in two steps.

Step 1: Linear Transformation

$$z^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}$$

Step 2: Non-linear Activation

$$h^{(l)} = \rho\left(z^{(l)}\right)$$

where:

- $W^{(l)}$is the weight matrix of layer l,
- $b^{(l)}$is the bias vector,
- $\rho(\cdot)$is the activation function such as ReLU, sigmoid, or tanh,
- $h^{(l)}$is the output of layer l,
- $h^{(0)} = x$, the input vector.

The overall network is a composition of such transformations:

$$\Phi(x) = W_L\rho\left(W_{L-1}\rho\left(\cdots\rho(W1x + b1)\cdots\right)+b_{L-1}\right) + b_L$$

This formulation shows that an MLP is built by stacking linear and non-linear transformations layer by layer.

Forward Propagation

Forward propagation is the process of passing input data through the network layer by layer. The input $x$is transformed using weights, biases, and activation functions until the final output $\hat{y}$is produced. In a feedforward neural network, information flows forward from the input to the output, and this process defines how predictions are generated.

For a typical MLP, the layer-wise computation can be written as:

$$h_1 = \rho(W_1x + b_1)$$
$$h_2 = \rho(W_2h_1 + b_2)$$
$$\hat{y} = g(W_Lh_{L-1} + b_L)$$

where $g(\cdot)$denotes the output activation function. In a classification task, $g(\cdot)$is usually softmax. The forward pass is also the stage at which the prediction is produced before the loss is computed.

Loss Function (Error Measurement)

To evaluate how well the model performs, a loss function is used. It measures the difference between the predicted output $\hat{y}$and the actual output y. The goal of training is to minimise this loss function so that predictions become closer to the true targets.

For regression problems, the least-squares loss is commonly used:

$$J(\theta) = \frac{1}{2} \| y - \hat{y} \|^2$$

For classification problems, cross-entropy loss is used. Since the Iris dataset in this experiment is a multi-class classification problem, categorical cross-entropy is appropriate:

$$J(\theta) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

where:

- $y_i$is the true one-hot encoded label,
- $\hat{y}_i$is the predicted probability for class i,
- $C$is the number of classes,
- θrepresents all model parameters.

This loss function quantifies how far the predicted probability distribution is from the true class distribution.

Backpropagation (Learning Mechanism)

Backpropagation is the core algorithm used to train an MLP. It computes how the loss changes with respect to each parameter in the network. More precisely, the objective is to compute gradients of the cost function $J(\theta)$with respect to the weights and biases. This is done using the chain rule of differentiation.

The gradients computed during backpropagation include:

$$\frac{\partial J(\theta)}{\partial W^{(l)}}, \frac{\partial J(\theta)}{\partial b^{(l)}}$$

The process can be described as follows:

1. Compute the error at the output layer.
2. Propagate the error backward through the hidden layers.
3. Compute the gradient of the loss with respect to each parameter.

These gradients are then used to update the model parameters during training.

Parameter Update (Optimisation)

Once gradients are computed, the parameters are updated using gradient descent:

$$\theta \leftarrow \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

where $\eta$is the learning rate. This update rule moves the parameters in the direction that reduces the loss function.

Common optimisers include:

- Stochastic Gradient Descent (SGD)
- RMSprop
- Adam

These optimisation methods differ in how they scale or adapt the updates, but they all aim to minimise the same cost function $J(\theta)$.

Optimisation Algorithms

Common optimisation algorithms used for training neural networks include Stochastic Gradient Descent (SGD), RMSprop, and Adam. These methods aim to minimise the cost function $J(\theta)$by iteratively updating the model parameters θ. Although all three algorithms optimise the same objective function, they differ in how they compute and scale parameter updates, which affects convergence speed and stability.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is one of the simplest and most widely used optimisation techniques. It updates the model parameters using the gradient of the loss function computed over a mini-batch of training samples. The update rule is given by

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t),$$

where $\eta$is the learning rate and $\nabla_\theta J(\theta_t)$represents the gradient of the cost function with respect to the parameters at iteration t. Although SGD is computationally efficient, it may converge slowly and can exhibit oscillations, particularly in regions where the loss surface is highly curved.

RMSprop

RMSprop is an adaptive learning rate method introduced by Geoffrey Hinton that improves upon SGD by scaling the parameter updates based on the magnitude of recent gradients. It maintains an exponentially weighted moving average of the squared gradients, defined as

$$E[g^2]_t = \rho \cdot E[g^2]_{t-1} + (1-\rho) \cdot \left(\nabla\theta J(\theta_t)\right)^2$$

where ρ is the decay rate (typically set to 0.9). The parameter update rule is then given by:

$$\theta_{t+1} = \theta_t - \left(\frac{\eta}{\sqrt{(E[g^2]_t + \varepsilon)}} \cdot \nabla\theta J(\theta_t) \right)$$

where ε is a small constant (e.g., 10^{-8}) added to ensure numerical stability. By normalising the gradient by the root mean square of recent gradients, RMSprop helps stabilise the learning process and is particularly effective when dealing with non-stationary objectives or recurrent networks.

Adam (Adaptive Moment Estimation)

Adam is an advanced optimisation algorithm that combines the benefits of momentum and adaptive learning rates. It computes both the first moment (mean) and the second moment (uncentred variance) of the gradients. The moment estimates are given by

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta J(\theta_t),$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta J(\theta_t))^2,$$

where β_1 and β_2 are decay rates (typically $\beta_1 = 0.9$ and $\beta_2 = 0.999$). Since m_t and v_t are initialised to zero, they are biased toward zero, especially in the early steps. These estimates are therefore bias-corrected as follows:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

The parameter update rule is then given by

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t.$$

Adam is widely used due to its efficiency, robustness, and ability to converge quickly in practice. The bias-correction step is essential for stable convergence in the initial training iterations.

Training Workflow

The entire training process consists of repeating the following steps:

- Forward propagation to compute predictions.
- Loss computation to measure prediction error.
- Backpropagation to compute gradients.
- Parameter update using an optimisation algorithm.

This cycle is repeated for multiple epochs until the model converges or reaches a predefined stopping condition, with the goal of progressively minimising the loss function J(θ). The overall MLP forward and backward propagation process is illustrated in Figure 2.

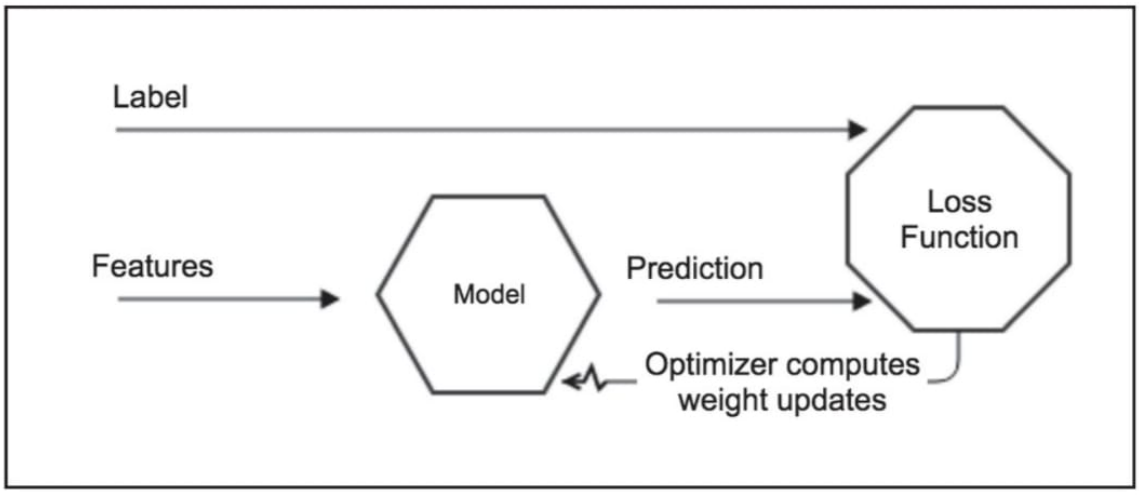

Figure 2- MLP Process both Forward and Backpropagation

(Source: Antonio Gulli, Sujit Pal, Deep Learning with Keras)

In a neural network, individual neuron outputs matter less than the collective behaviour of the weights in each layer. The network adjusts its internal weights so that prediction accuracy increases over successive epochs. Using appropriate features and high-quality labels is fundamental for reducing bias and improving learning.

Generalisation, Overfitting, and Underfitting

The ultimate goal of training is not only to perform well on the training set but also to make accurate predictions on unseen data. This ability is called generalisation. A model generalises well when it captures the underlying structure of the data rather than memorising the training examples. In such a case, both training and testing performance remain high, and the gap between them is small.

Overfitting occurs when the model learns the training data too well, including noise and specific details, and performs poorly on unseen data. It is characterised by high training accuracy and low testing accuracy.

Underfitting occurs when the model is too simple to learn the underlying pattern in the data. In this case, both training and testing accuracy remain low.

Challenges in Training Deep Networks

Training deep neural networks presents several challenges. One of the most important is the vanishing gradient problem. During backpropagation, gradients are propagated backward through multiple layers using the chain rule. When activation functions such as sigmoid or tanh are used, their derivatives may become very small, causing gradients to shrink as they move backward. As a result, earlier layers receive very small updates and learning becomes slow.

The exploding gradient problem is the opposite case, where gradients become excessively large during backpropagation. This can lead to unstable updates and divergence during training. Techniques such as careful initialisation, suitable learning rates, and gradient clipping are commonly used to reduce this problem.

Another challenge is hyperparameter sensitivity. The performance of an MLP depends strongly on the learning rate, number of layers, number of neurons, activation functions, batch size, and number of epochs. Poor hyperparameter choices can lead to unstable training, overfitting, or underfitting.

Merits of Feedforward Neural Network (MLP):

- **Scalability:** The number of hidden layers and neurons can be adjusted to match problem complexity.
- **Performance on tabular data:** For many structured datasets, MLPs can outperform more complex models due to their simplicity and ability to learn direct features.
- **Universal function approximation:** MLPs can approximate virtually any continuous function, enabling them to model complex, non-linear relationships.

Demerits of Feedforward Neural Network (MLP):

- **Sensitivity to hyperparameters:** Performance depends heavily on choices such as number of layers, units, learning rate, and activation functions.
- **Overfitting:** MLPs can memorise training data and generalise poorly, especially with small datasets.
- **Gradient issues:** Very deep networks may encounter vanishing or exploding gradients, making training difficult.
- **Data requirements:** Large amounts of labelled data are often necessary to train effectively.

Code & Result:

[ ]:
```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname,  filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
#       kagglehub.dataset_download('<owner>/<dataset-slug>')
```

Step-1: Import Libraries

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelBinarizer
from sklearn.metrics import classification_report, confusion_matrix

import tensorflow as tf
from tensorflow.keras import Model
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import plot_model
from IPython.display import Image, display
```

```
2026-05-26 06:24:48.332667: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1779776688.730821    57 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1779776688.845828    57 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1779776689.848222    57 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779776689.848263    57 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779776689.848266    57 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779776689.848268    57 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
```

Step-2: Load Dataset

[ ]:
```
FILE_PATH = "/kaggle/input/datasets/organizations/uciml/iris/Iris.csv"
data = pd.read_csv(FILE_PATH)
if "Id" in data.columns:
    data = data.drop("Id", axis=1)

print("Shape:", data.shape)
display(data.head())
print(data["Species"].value_counts())
```

```
Shape: (150, 5)
```

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species

0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

Step-3: Intializing Parameters

[ ]:
```
X = data.drop("Species", axis=1).values
y_species = data["Species"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

binarizer = LabelBinarizer()
y_encoded = binarizer.fit_transform(y_species)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2,
    random_state=42, stratify=y_encoded)

print("Train Shape:", X_train.shape)
print("Test Shape:", X_test.shape)
print("Classes:", binarizer.classes_)
```

```
Train Shape: (120, 4)
Test Shape: (30, 4)
Classes: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

Step-4: Model Building

[ ]:
```
input_dim = 4

inputs = Input(shape=(input_dim,))
x1 = Dense(10, activation="relu")(inputs)
x2 = Dense(8, activation="relu")(x1)
outputs = Dense(3, activation="softmax")(x2)

model = Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
```

```
)

model.summary()
```

```
I0000 00:00:1779777083.078318     57 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB memory:  -> device:
0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1779777083.084600     57 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:1 with 13757 MB memory:  -> device:
1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 4)	0
dense (Dense)	(None, 10)	50
dense_1 (Dense)	(None, 8)	88
dense_2 (Dense)	(None, 3)	27

Total params: 165 (660.00 B)

Trainable params: 165 (660.00 B)

Non-trainable params: 0 (0.00 B)

Step-5: Model Training

[ ]:
```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.1,
    verbose=0
)

model.compile(
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01
    ),
```

```
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

print("Training Completed!")
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1779777095.442841   149 service.cc:152] XLA service 0x7a4974006610 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
I0000 00:00:1779777095.442879   149 service.cc:160]   StreamExecutor device (0): Tesla T4, Compute Capability 7.5
I0000 00:00:1779777095.442882   149 service.cc:160]   StreamExecutor device (1): Tesla T4, Compute Capability 7.5
I0000 00:00:1779777095.878490   149 cuda_dnn.cc:529] Loaded cuDNN version 91002
I0000 00:00:1779777096.865201   149 device_compiler.h:188] Compiled cluster using XLA!  This line is logged at most once for the lifetime of the process.

Training Completed!
```

Step-6: Model Evaluation

```
[ ]: y_true = binarizer.inverse_transform(y_test)
y_pred = binarizer.inverse_transform(
    model.predict(X_test, verbose=0)
)

report = pd.DataFrame(
    classification_report(y_true, y_pred, output_dict=True)
).transpose()

display(report)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
plt.imshow(cm, cmap="Blues")

plt.title("Confusion Matrix")
plt.colorbar()

for i in range(len(cm)):
    for j in range(len(cm[0])):
        plt.text(j, i, cm[i, j],
                 ha='center', va='center',
```

```
                  color='black')

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

	precision	recall	f1-score	support
Iris-setosa	1.000000	0.9	0.947368	10.0
Iris-versicolor	0.888889	0.8	0.842105	10.0
Iris-virginica	0.833333	1.0	0.909091	10.0
accuracy	0.900000	0.9	0.900000	0.9
macro avg	0.907407	0.9	0.899522	30.0
weighted avg	0.907407	0.9	0.899522	30.0